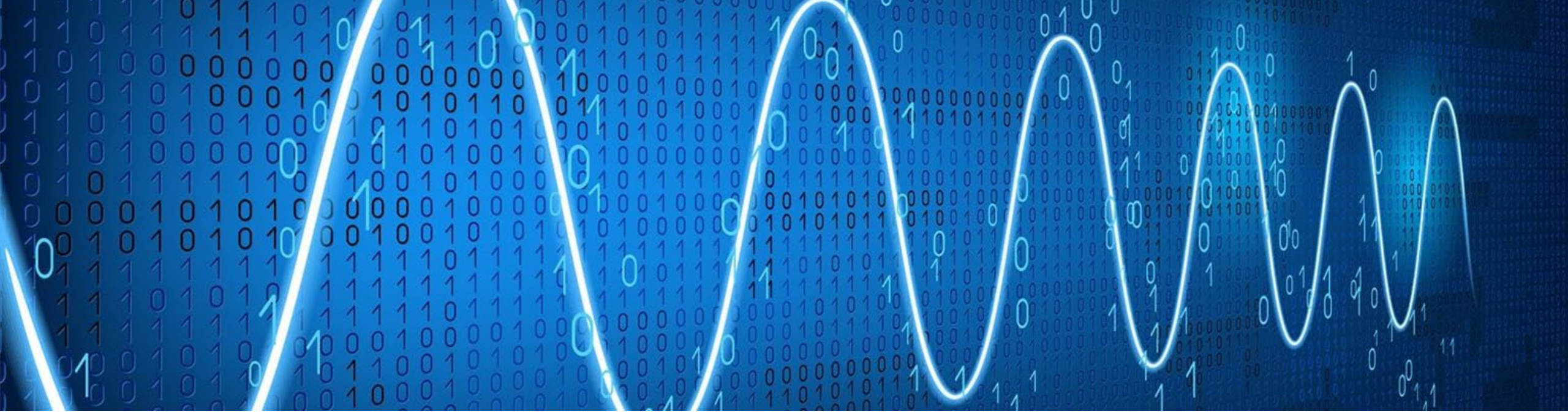




Lecture 13:

Fast Fourier Transform (FFT)

Azra Abtahi

Email: azra.abtahi-fahliani@mau.se

Faculty of Technology and Society Department of Computer Science and Media
Technology Malmö University

Where We Are in the Course

- Analog Signal Processing, Introduction to digital signal processing (Lectures 2-6)
- LTI System (Lectures 7 and 9)
- Z-Transform, System Function, Frequency Response (Lectures 8)
- FIR Filters (Lecture 8)
- IIR Filters, Linear Phase Filters (Lecture 10)
- Implementation Aspects (Lecture 11)
- CTFT, DTFT, **DFT** (Lecture 12)
- ***FFT*** (Lecture 13)

Lecture 14: Sample Questions

Outline

- **Reminder on DFT**
- ***Twiddle Factors***
- ***Fast Fourier Transform:***
 - ***Recursive Algorithm***
 - ***Iterative Algorithm***

Reminder on DFT

Discrete-Fourier Transform (DFT)

Discrete Time $\leftrightarrow$ Discrete Frequency

$$x[n] = x(nT_s) \overset{\text{DFT}}{\longleftrightarrow} X[k]$$

N-Point DFT:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad \text{for } k = 0, \dots, N - 1$$

Inverse DFT

If the length of $x[n]$ is denoted by L and $L \leq N$:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j\frac{2\pi}{N}kn} \quad \text{for } n = 0, \dots, N - 1$$

Example of DFT

- $x[n] = [1 \ 2 \ 3]$, 3-Point DFT?

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad \text{for } k = 0, \dots, N-1, N = 3$$

$$X[k=0] = \sum_{n=0}^2 x[n] e^{-j\frac{2\pi}{3}0n} = \sum_{n=0}^2 x[n] = 6$$

$$\begin{aligned} X[k=1] &= \sum_{n=0}^2 x[n] e^{-j\frac{2\pi}{3}n} \\ &= x[0]e^{-j\frac{2\pi}{3}0} + x[1]e^{-j\frac{2\pi}{3}} + x[2]e^{-j\frac{2\pi}{3}2} = -1.5 + j0.866 \end{aligned}$$

$$\begin{aligned} X[k=2] &= \sum_{n=0}^2 x[n] e^{-j\frac{2\pi}{3}2n} \\ &= x[0]e^{-j\frac{2\pi}{3}0} + x[1]e^{-j\frac{2\pi}{3}} + x[2]e^{-j\frac{2\pi}{3}2} = -1.5 + j0.866 \end{aligned}$$

Discussion Time I

How many complex multiplications and additions are required when computing the DFT directly from its definition?

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad \text{for } k = 0, \dots, N - 1$$



Discussion Time I

How many complex multiplications and additions are required when computing the DFT directly from its definition?

Multiplications = N^2

Additions = $N(N - 1)$

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad \text{for } k = 0, \dots, N - 1$$

DFT has a complexity of $O(N^2)$

The **order of complexity** describes how the running time grows as input size N increases.

Twiddle Factors

Twiddle Factors

$$w_N = e^{-j\frac{2\pi}{N}} \rightarrow w_N^{nk}: \textbf{Twiddle Factors}$$

N-Point DFT:

$$X[k] = \sum_{n=0}^{N-1} x[n] w_N^{nk} \quad \text{for } k = 0, \dots, N - 1$$

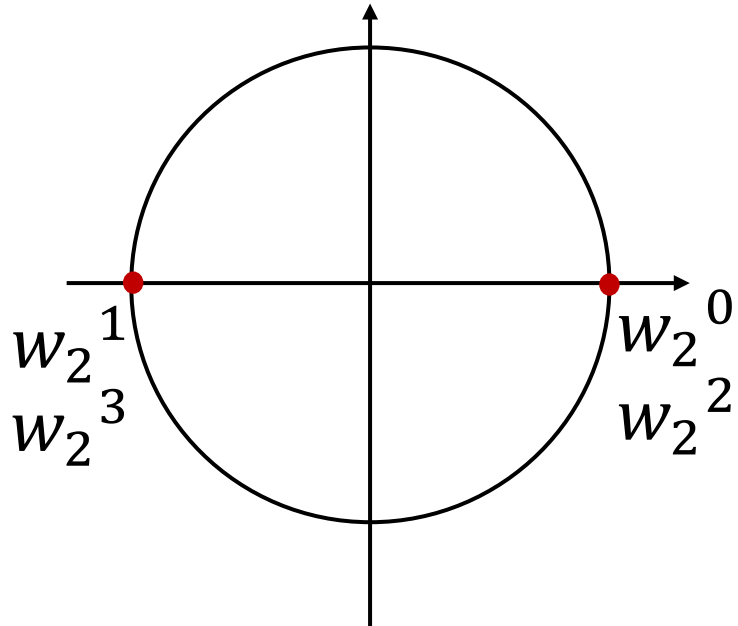
Inverse DFT

If the length of $x[n]$ is denoted by L and $L \leq N$:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] w_N^{-nk} \quad \text{for } n = 0, \dots, N - 1$$

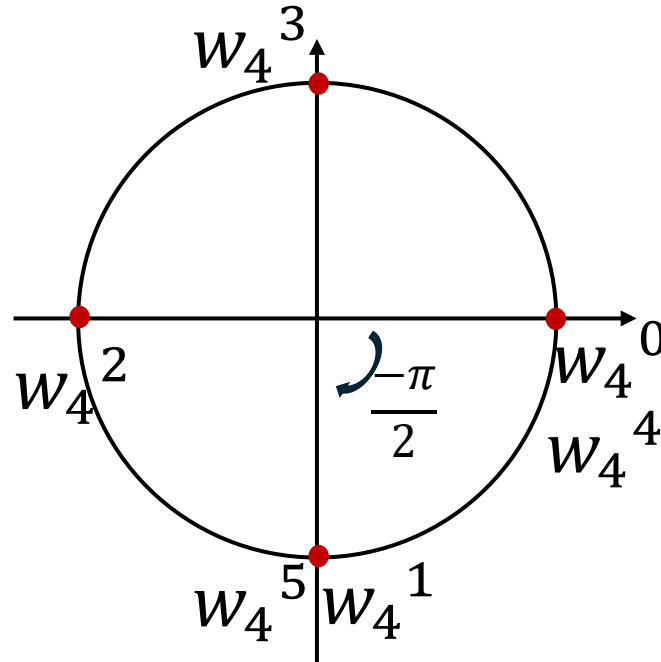
Twiddle Factors

$N = 2$



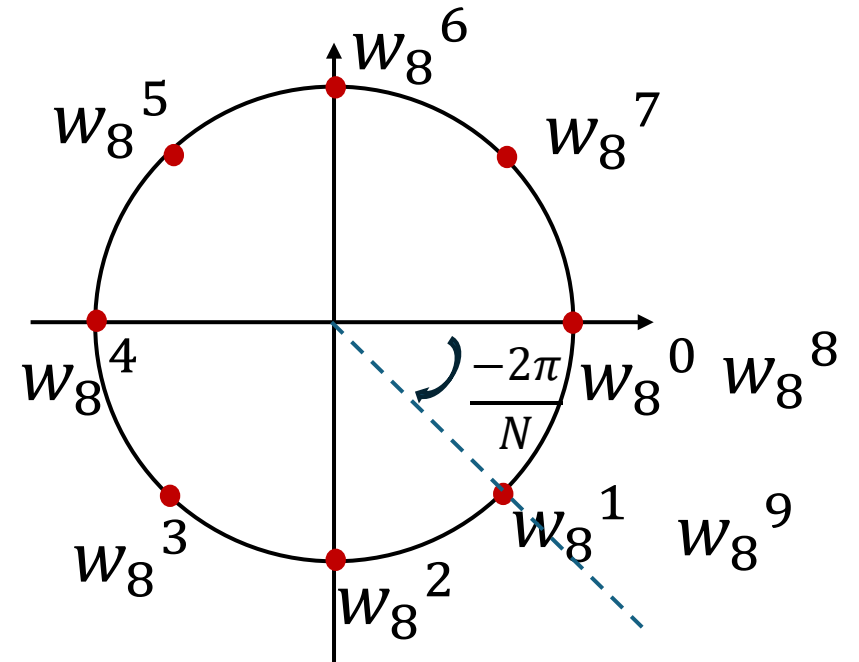
$$w_2 = e^{-j\frac{2\pi}{N}} = e^{-j\frac{2\pi}{2}} = e^{-j\pi} = -1$$

$N = 4$



$$w_4 = e^{-j\frac{\pi}{2}}$$

$N = 8$



$$w_8 = e^{-j\frac{\pi}{4}}$$

Twiddle Factors' Properties

- P1: $w_N^{N+\tilde{k}} = w_N^{\tilde{k}}$

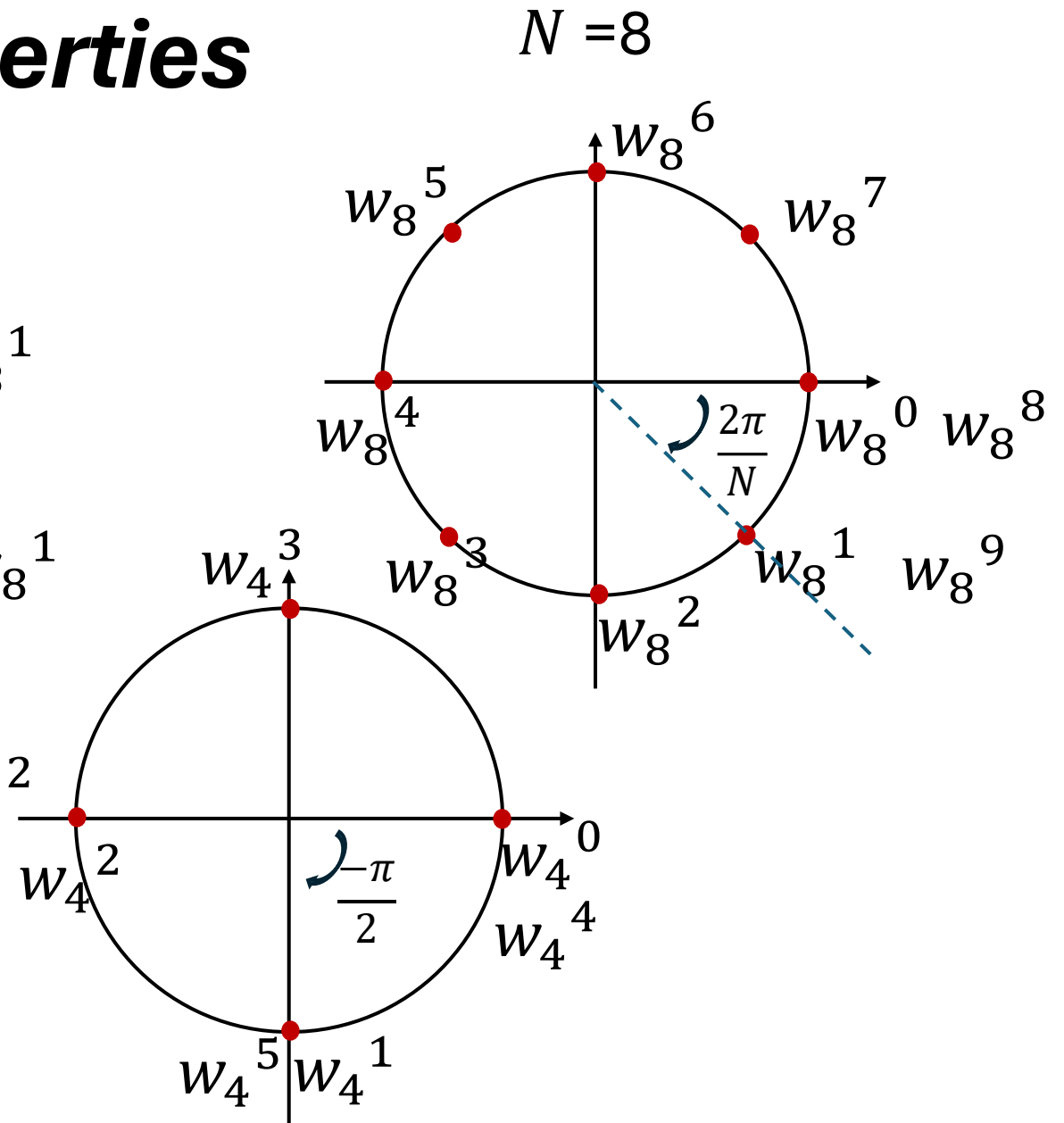
Example: $N = 8, \tilde{k} = 1: w_8^9 = w_8^1$

- P2: $w_N^{N/2+\tilde{k}} = -w_N^{\tilde{k}}$

Example: $N = 8, \tilde{k} = 1: w_8^5 = -w_8^1$

- P3: $w_{(N/2)}^{(\tilde{k}/2)} = w_N^{\tilde{k}}$

Example: $N = 8, \tilde{k} = 2: w_4^1 = w_8^2$



Example

- $N = 4$

$$X[k] = \sum_{n=0}^3 x[n](-j)^{nk} \quad \text{for } k = 0, \dots, 3$$

$$X[0] = \sum_{n=0}^3 x[n] \cdot 1 = x[0] + x[1] + x[2] + x[3]$$

$$X[1] = \sum_{n=0}^3 x[n] w_4^n = x[0] - jx[1] - x[2] + jx[3]$$

$$X[2] = \sum_{n=0}^3 x[n] w_4^{2n} = x[0] - x[1] + x[2] - x[3]$$

$$X[3] = \sum_{n=0}^3 x[n] w_4^{3n} = x[0] + jx[1] + x[2] - jx[3]$$

$w_4^{\tilde{k}}$	$\tilde{k}$
1	0
-j	1
-1	2
j	3
1	4
...	...

FFT Algorithms

Recursive Algorithm

- $N = \min \{2^m | 2^m \geq L\}$
- Follows a divide-and-conquer strategy

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}$$
$$= \sum_{n=0}^{\frac{N}{2}-1} x[2n] e^{-j \frac{2\pi}{N} k(2n)} + \sum_{n=0}^{\frac{N}{2}-1} x[2n+1] e^{-j \frac{2\pi}{N} k(2n+1)}$$

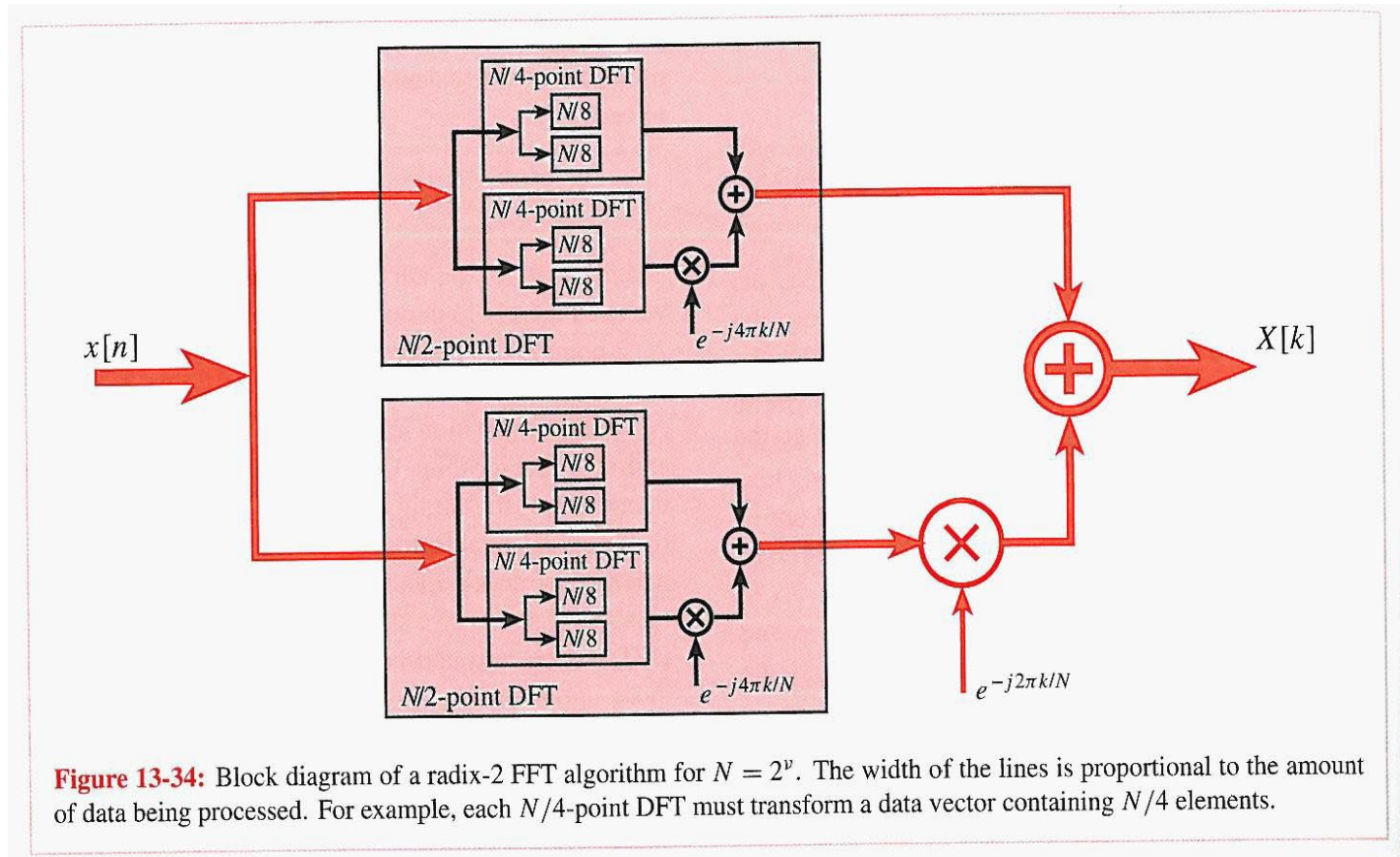
$$N = 2^m$$

$$= \underbrace{\sum_{n=0}^{2^{m-1}-1} x[2n] e^{-j \frac{2\pi}{2^{m-1}} k(n)}}_{\text{N/2-point DFT of even-indexed samples}} + \underbrace{\sum_{n=0}^{2^{m-1}-1} x[2n+1] e^{-j \frac{2\pi}{2^{m-1}} kn}}_{\text{N/2-point DFT of odd-indexed samples}} (e^{-j \frac{2\pi}{2^m} k})$$

$\frac{N}{2}$ –point DFT of even-indexed samples

$\frac{N}{2}$ –point DFT of odd-indexed samples

Recursive Algorithm



Iterative Algorithm

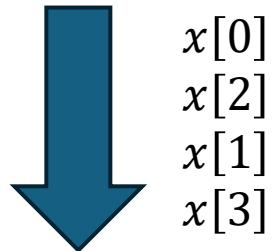
- To reduce memory usage and enable faster computation (since recursive algorithms require waiting for recursive calls to complete).
- $N = \min\{2^m \mid 2^m \geq L\}$

Iterative Algorithm

1. Recover inputs using Bit-Reversal:

Write indexes $k = 0, \dots, N - 1$ in binary and reverse it to find the order of $x[n]$.

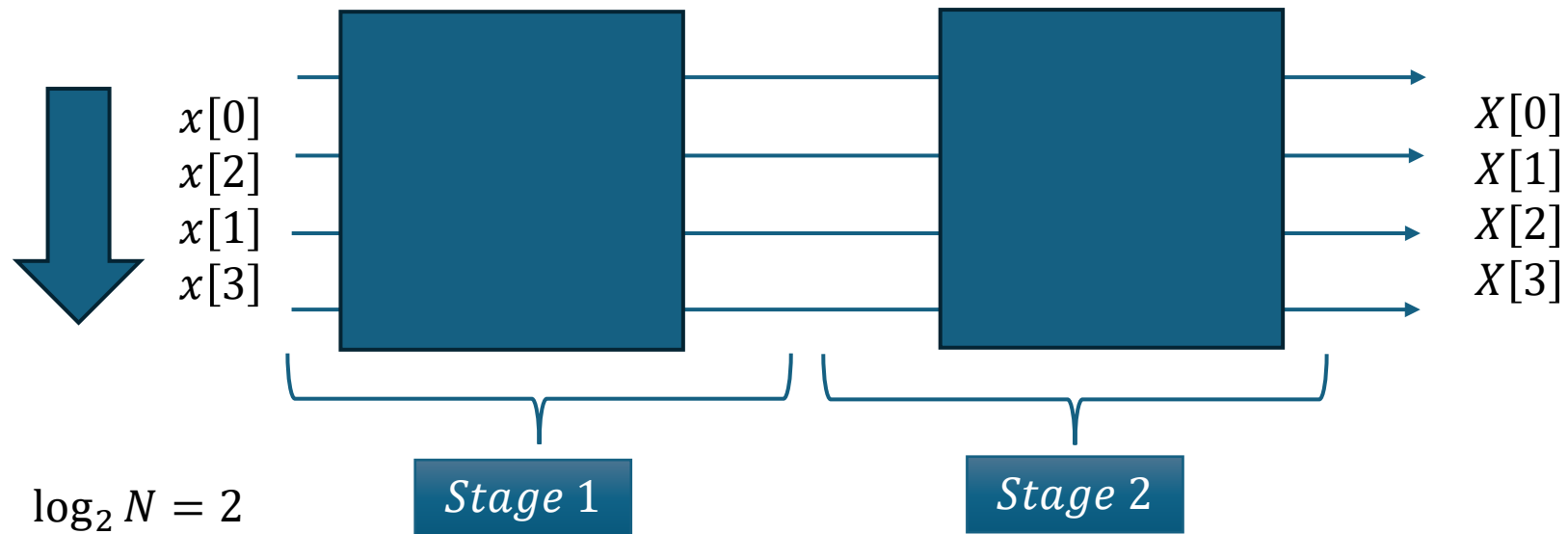
	0	1	2	3
k	00	01	10	11
n	00	10	01	11
	0	2	1	3



Iterative Algorithm

2. For $i = 1: \log_2 N$

We have $\log_2 N$ stages/iterations to calculate DFT

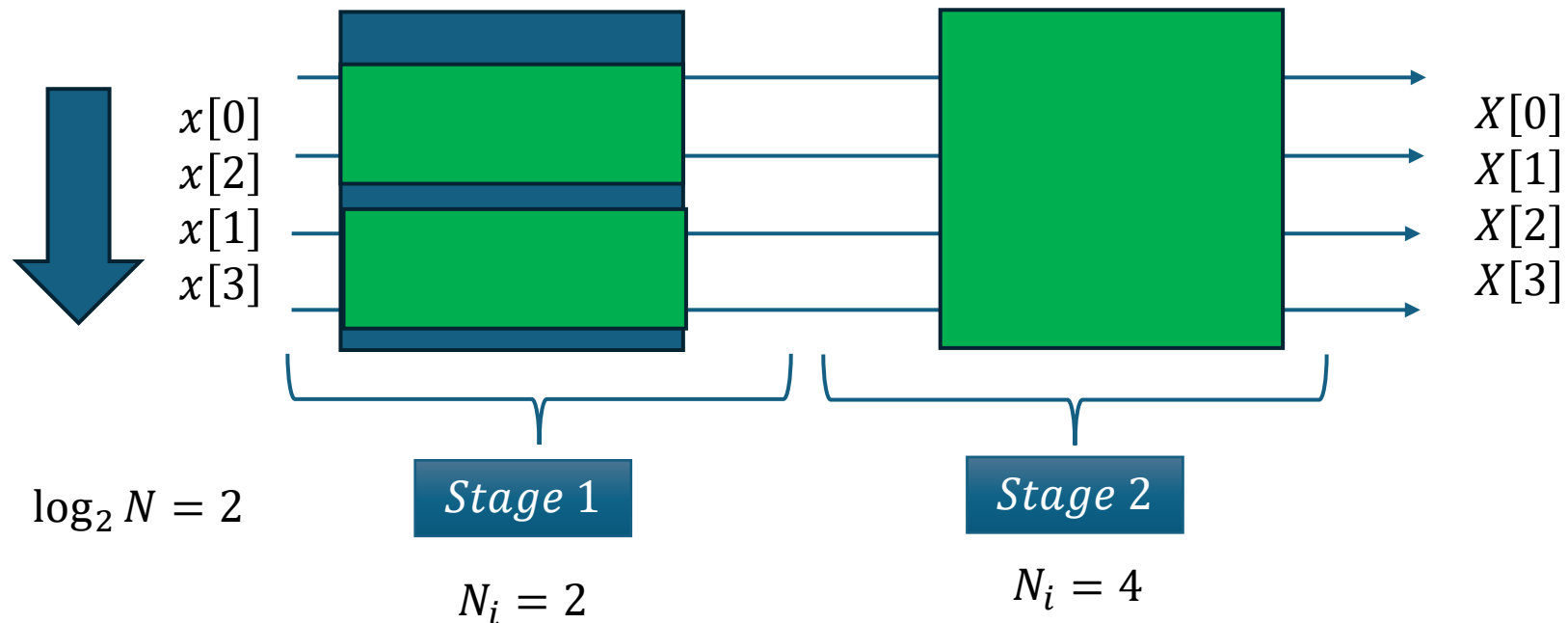


Iterative Algorithm

2. For $i = 1: \log_2 N$

We have $\log_2 N$ stages/iterations to calculate DFT

a) Divide the inputs to stage i to groups with length $N_i = 2^i$

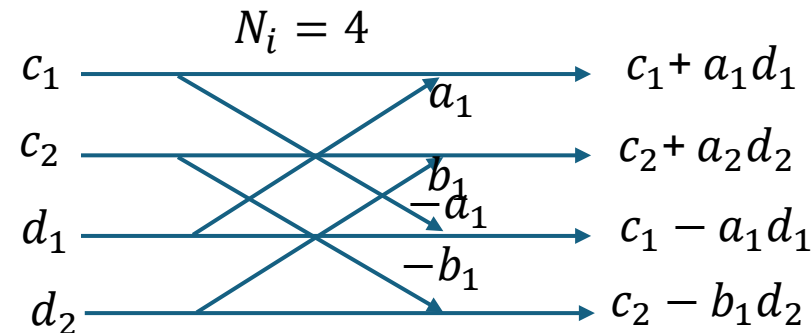
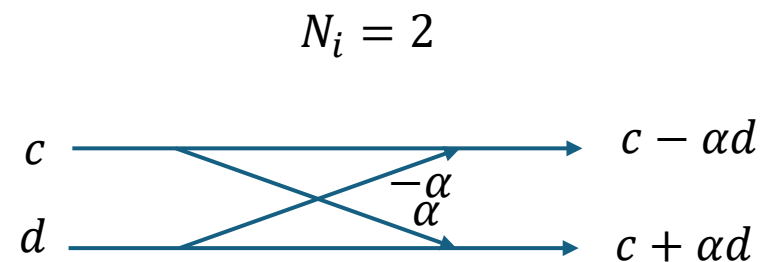


Iterative Algorithm

2. For $i = 1: \log_2 N$

- a) Divide the inputs to stage i to groups with length $N_i = 2^i$ and combine the group members using butterfly connections with the length N_i .

In the h th group the input of stage i , $\tilde{x}[l + hN_i]$, is combined with $\tilde{x}[l + hN_i + \frac{N_i}{2}]$.

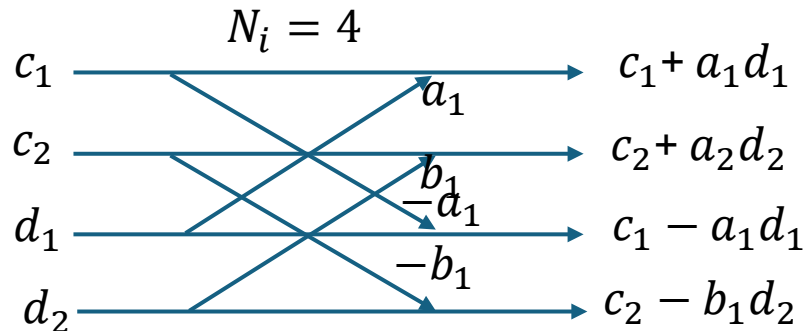
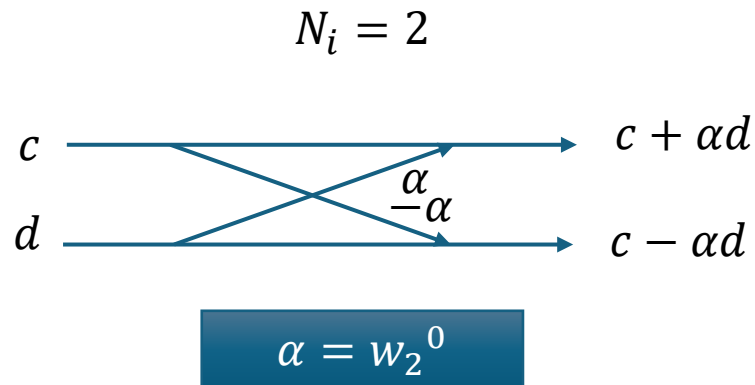


Iterative Algorithm

2. For $i = 1: \log_2 N$

We have $\log_2 N$ stages/iterations to calculate DFT

- a) Divide the inputs to stage i to groups with length $N_i = 2^i$ and combine the group members using butterfly connections with the length N_i .



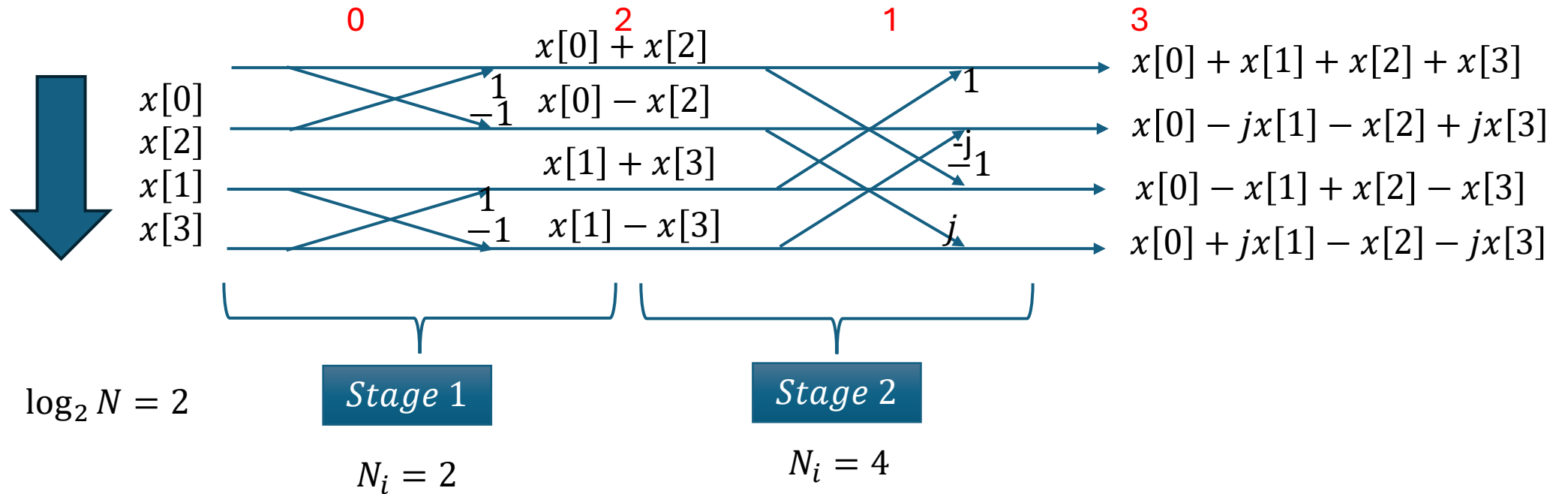
- b) The connections' weights are Twiddle factors of N_i .

$$\begin{aligned} a_1 &= w_4^0 \\ b_1 &= w_4^1 \end{aligned}$$

$$\begin{aligned} -a_1 &= w_4^2 \\ -b_1 &= w_4^3 \end{aligned}$$

Example $N = 4$

	0	1	2	3
k	00	01	10	11
n	00	10	01	11



Discussion Time II

In the iterative FFT algorithm of size N , what are the total number of complex multiplications required?



Iterative Algorithm

$$W_{N_i}^{\frac{N_i}{2} + \tilde{k}} = -W_{N_i}^{\tilde{k}}$$

Complex Multiplications:

$$\frac{N}{2}$$

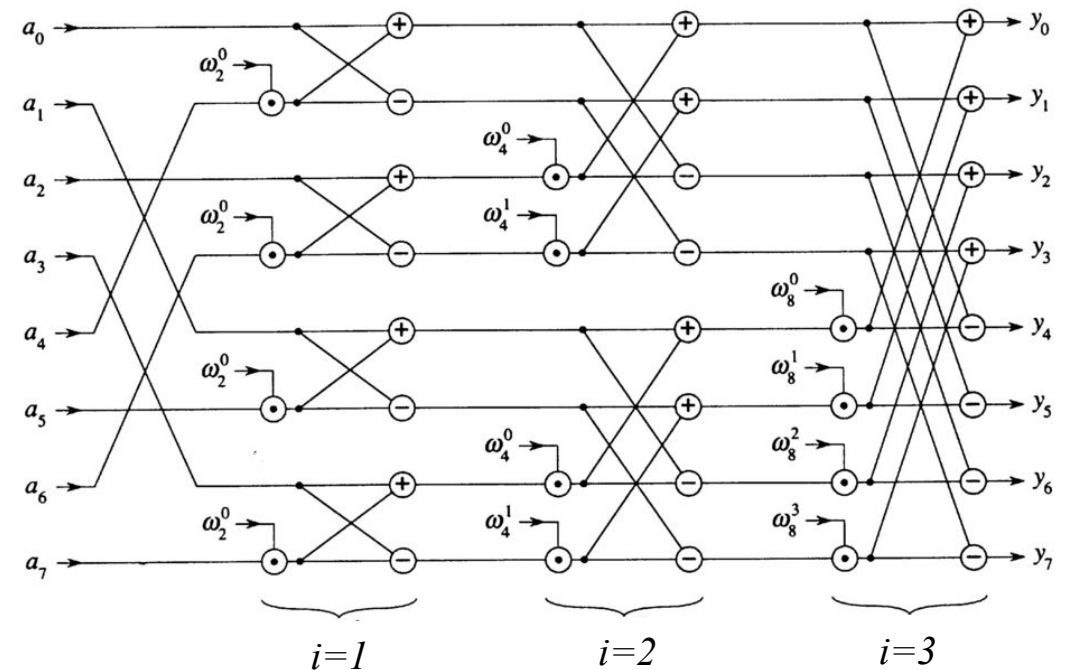


$$\log_2 N$$

$$N = 2^{10};$$

$$N^2 = 1,048,576$$

$$\frac{N}{2} \log_2 N = 5120$$



FFT has a complexity of $O(\frac{N}{2} \log_2 N)$